KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Personal Contact Directory with Search and Update Features

Submitted by

ROSHINI KUMARI P

25BCS142

Under the Guidance of

MR ARAVIND

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **2**

DECLARATION

I hereby declare that the project entitled "**Personal Contact Directory with Search and Update Features" submitted** in partial fulfillment of the requirements for the course **BSc CS** is a **record of my original work** carried out under the guidance of **MR ARAVIND** Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

ROSHINI KUMARI P

CERTIFICATE

This is to certify that the project entitled **"Personal Contact Directory with Search and Update Features"** is a **bona fide work** carried out by **ROSHINI KUMARI P** I 'B'**(2025 - 2028)** , BSc CS, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **BSc CS**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date:10.4.26

Submitted for the Viva-Voce Examination held on **10.4.26**

Guide:	**Head of the Department**
(Technical Trainer, KGCAS)	(Technical Head, KGCAS)
MR.ARAVIND	MR.BOOPALAN SUNDARMOORTHY

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	**3**
1.1	Objective	3
1.2	Overview	3
1.3	Features	4
2	**Program Analysis**	5
2.1	Problem Definition	5
2.2	Project Overview	5
2.3	Scopes and Limitations	6
3	**System Design**	**7**
3.1	Modules	7
4	**Methodology**	**10**
4.1	Problem Analysis	10
4.2	System Design	10
4.3	Program Development	11
4.4	Testing and Validation	11
4.5	Result and Documentation	12
4.6	Algorithm	12
4.7	Flowchart	13
5	**Source Code**	**14**
6	**Input and Output Screens**	**19**
7	**Discussion and Conclusion**	**21**
7.1	Discussion	21
7.2	Conclusion	22
8	**References**	**23**

ABSTRACT

The Personal Contact Directory with Search and Update Features is a Java-based application designed to manage personal contacts efficiently. In many cases, people store contact details manually or in unorganized formats, which makes searching and updating information difficult. This project aims to provide a simple and structured way to store, search, and update contact details.

The system allows users to add new contacts, search for existing contacts by name, update contact details, and delete contacts when required. The application is developed using Core Java concepts such as classes, objects, arrays, loops, conditional statements, and file handling.

The main objective of this system is to simplify contact management, improve data organization, and provide quick access to contact details. This project demonstrates the practical implementation of Java programming in developing a real-world application.]

1. INTRODUCTION

In today's digital world, managing personal contacts efficiently is very important. Many people still rely on manual methods such as notebooks or unorganized digital lists to store contact information. This often leads to difficulties in searching, updating, and maintaining accurate records.

With the help of technology, a computerized contact management system can solve these problems effectively. A Personal Contact Directory allows users to store and manage contact details such as name, phone number, and address in an organized manner.

This project is a Java-based application that helps users perform operations like adding new contacts, searching contacts, updating existing information, and deleting unwanted records. The system ensures that contact details are easily accessible and manageable.

The application is developed using Core Java concepts and can be implemented as a console-based program or using Java Swing for a graphical interface.

1.1 Objective

The main objective of the Personal Contact Directory is to develop an efficient system for managing personal contacts.

Specific objectives include:

To design a Java-based contact management system

To store and manage contact details efficiently

To provide search functionality for quick access

To allow updating and deleting of contacts

To reduce manual effort in managing contacts

To demonstrate Core Java concepts

1.2 Overview

The Personal Contact Directory is a menu-driven Java application that allows users to manage their contacts digitally.

Users can:

Add new contacts

Search contacts by name

Update existing contact details

Delete contacts

The system stores contact information in an organized way and provides fast access using search functionality.

1.3 Features

• Add Contact – Add new contact details

• Search Contact – Search by name

• Update Contact – Modify existing details

• Delete Contact – Remove unwanted contacts

• Display Contacts – View all saved contacts

• User-Friendly Interface – Simple menu-driven system

2.PROGRAM ANALYSIS

Program analysis is a crucial phase in software development where the problem is studied in detail and the system requirements are clearly understood. This stage helps in identifying the limitations of the existing system and designing an improved computerized solution.

In this project, the analysis focuses on the difficulties faced in managing personal contacts manually and how a Java-based application can solve these issues efficiently.

2.1 Problem Definition

In daily life, people store contact details in notebooks, mobile phones, or unorganized digital files. Managing contacts in such ways leads to several problems, especially when the number of contacts increases.

Some of the major problems are:

• Difficulty in searching for a specific contact quickly

• Time-consuming process to update contact details

• Chances of losing important contact information

• Data duplication due to repeated entries

• Lack of proper organization and structure

• No easy way to delete unwanted or outdated contacts

• Manual systems are not efficient for large data

• High chances of human error while writing or updating details

For example, if a user wants to update a phone number, they may need to search manually through many records, which wastes time and effort.

Therefore, there is a need for an automated system that can store, search, update, and delete contact information efficiently.

The Personal Contact Directory is designed to overcome these problems by providing a simple, fast, and user-friendly solution.

2.2 Project Overview

The Personal Contact Directory is a Java-based application that helps users manage their contacts digitally.

The system provides a menu-driven interface where users can perform different operations on contact data. Each contact includes details such as name, phone number, and address.

The system performs the following functions:

• Adding new contact details

• Searching contacts using name

• Updating existing contact information

• Deleting unwanted contacts

• Displaying all stored contacts

The system is designed using Core Java concepts such as classes, objects, arrays, loops, and conditional statements. It can be implemented as a console-based application or enhanced into a graphical user interface.

This project provides a simple solution for personal contact management and demonstrates practical programming skills.

2.3 Scope and Limitations

Scope:

The Personal Contact Directory system can be used for:

• Managing personal contacts efficiently

• Storing and organizing contact information

• Quick searching and updating of data

• Learning and implementing Java programming concepts

• Small-scale applications without the need for complex systems

The system can be useful for:

• Students

• Individuals

• Small

Limitations:

Despite its advantages, the system has some limitations:

• Limited storage if database is not used

• Data may not be permanent without file handling

• No online or cloud access

• Limited security features

- Console-based interface may not be visually attractive
- Not suitable for large-scale applications

3.SYSTEM DESIGN

System design is an important phase in software development where the overall structure of the system is planned before implementation. In this stage, the system architecture, components, and workflow are clearly defined. A well-designed system ensures smooth functioning, easy maintenance, and better performance.

The Personal Contact Directory with Search and Update Features is designed as a menu-driven application. The system allows users to perform operations such as adding, searching, updating, deleting, and displaying contact details in an organized manner.

The system is divided into different modules, where each module performs a specific function. This modular approach makes the program easy to understand, develop, and maintain.

3.1 Modules

The main modules of the Personal Contact Directory are explained below:

1. Add Contact Module

This module is responsible for adding new contact details into the system.

Functions of this module include:

• Accepting user input such as name, phone number, and address

• Validating input data to avoid errors

• Storing contact details in an array or list

• Ensuring no duplicate entries are added

This module helps in building the contact database.

2. Search Contact Module

This module allows users to search for a contact easily.

Functions include:

• Accepting the name (or keyword) to search

• Comparing input with stored contact data

• Displaying matching contact details

• Showing message if contact is not found

This module reduces time and effort in finding specific contacts.

3. Update Contact Module

This module is used to modify existing contact details.

Functions include:

• Searching for the contact to be updated

• Allowing the user to edit details like phone number or address

• Saving the updated information

• Ensuring correct data replacement

This helps keep contact information accurate and up-to-date.

4. Delete Contact Module

This module removes unwanted or outdated contact details.

Functions include:

• Searching for the contact to delete

• Confirming deletion action

• Removing contact from storage

• Adjusting remaining records properly

This helps maintain a clean and organized contact list.

5. Display Contacts Module

This module displays all stored contacts.

Functions include:

• Showing complete list of contacts

• Displaying details in a structured format

• Ensuring readability for the user

This helps users view all stored information at once.

6. Data Storage Module

This module manages how contact data is stored.

Functions include:

• Storing data in arrays or ArrayList

• Managing memory efficiently

• (Optional) Saving data using file handling for permanent storage

This module ensures proper organization and accessibility of data.

7. User Interface Module

This module provides interaction between user and system.

Functions include:

• Displaying menu options

• Taking user input

• Guiding user through different operations

• Showing results and messages clearly

This ensures the system is easy to use and user-friendly.

8. Exit Module

This module allows the user to safely terminate the program.

Functions include:

• Providing exit option in menu

• Ending the program execution

• Ensuring no data loss before exit

This detailed system design helps in understanding how each part of the application works together to form a complete and efficient Personal Contact Directory system.

4. METHODOLOGY

Methodology explains the step-by-step process followed to develop the Personal Contact Directory system. It ensures that the project is completed in a structured and systematic manner.

The development of this project includes the following stages:

• Problem Analysis

• System Design

• Program Development

• Testing and Validation

• Result and Documentation

4.1 Problem Analysis

In this phase, the problems of manual contact management are studied carefully.

The main issues identified are:

• Difficulty in searching contacts quickly

• Time-consuming updates

• Poor organization of data

• Risk of losing important information

Based on these issues, system requirements were defined to develop an efficient contact management system.

4.2 System Design

In this stage, the structure of the system is planned.

The system is divided into modules such as:

• Add Contact

• Search Contact

• Update Contact

• Delete Contact

• Display Contacts

This makes the system easy to develop and understand.

4.3 Program Development

In this phase, the actual coding of the project is done using Java.

Steps followed:

• Creating classes and objects

• Writing logic for menu-driven program

• Implementing contact operations

• Handling user input

• Displaying output properly

Core Java concepts used:

• Classes and Objects

• Arrays / ArrayList

• Loops (for, while)

• Conditional statements (if-else, switch)

4.4 Testing and Validation

Testing ensures that the program works correctly without errors.

Different test cases are used:

Test Case

Expected Result

Add Contact

Contact stored correctly

Search Contact

Correct contact displayed

Update Contact

Data updated successfully

Delete Contact

Contact removed

Invalid Input

Error handled properly

This ensures the system is reliable and accurate.

4.5 Result and Documentation

After testing, the final results are observed.

The system successfully:

• Stores contact details

• Searches contacts quickly

• Updates and deletes records

• Displays all contacts

Documentation is prepared to explain the working of the system clearly.

4.6 Algorithm

Algorithm for Personal Contact Directory:

Start the program

Display menu options

Ask user to choose an option

If choice = Add Contact

→ Enter contact details

→ Store contact

If choice = Search Contact

→ Enter name

→ Display result

If choice = Update Contact

→ Search contact

→ Modify details

If choice = Delete Contact

→ Search contact

→ Delete it

If choice = Display Contacts

→ Show all contacts

Repeat steps until user selects Exit

End program

4.7 Flowchart

Flowchart Steps:

Start

↓

Display Menu

↓

User Choice

↓

Add / Search / Update / Delete / Display

↓

Perform Operation

↓

Back to Menu

↓

End

5. SOURCE CODE

```
import java.util.*;


class Contact {
   String name;
   String phone;
   String address;


   Contact(String name, String phone, String address) {
      this.name = name;
      this.phone = phone;
      this.address = address;
   }
}


public class ContactDirectory {
   static ArrayList<Contact> contacts = new ArrayList<>();


   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int choice;


      do {
         System.out.println("\n--- CONTACT DIRECTORY ---");
         System.out.println("1. Add Contact");
         System.out.println("2. Search Contact");
         System.out.println("3. Update Contact");
         System.out.println("4. Delete Contact");
```

```
System.out.println("5. Display All Contacts");
System.out.println("6. Exit");
System.out.print("Enter your choice: ");
choice = sc.nextInt();
sc.nextLine();

switch (choice) {
  case 1:
    addContact(sc);
    break;
  case 2:
    searchContact(sc);
    break;
  case 3:
    updateContact(sc);
    break;
  case 4:
    deleteContact(sc);
    break;
  case 5:
    displayContacts();
    break;
  case 6:
    System.out.println("Exiting...");
    break;
  default:
    System.out.println("Invalid choice!");
}
```

```
    } while (choice != 6);
}

static void addContact(Scanner sc) {
    System.out.print("Enter Name: ");
    String name = sc.nextLine();
    System.out.print("Enter Phone: ");
    String phone = sc.nextLine();
    System.out.print("Enter Address: ");
    String address = sc.nextLine();

    contacts.add(new Contact(name, phone, address));
    System.out.println("Contact added successfully!");
}

static void searchContact(Scanner sc) {
    System.out.print("Enter name to search: ");
    String name = sc.nextLine();
    boolean found = false;

    for (Contact c : contacts) {
        if (c.name.equalsIgnoreCase(name)) {
            System.out.println("Name: " + c.name);
            System.out.println("Phone: " + c.phone);
            System.out.println("Address: " + c.address);
            found = true;
        }
    }
```

```
    if (!found) {
        System.out.println("Contact not found!");
    }
}

static void updateContact(Scanner sc) {
    System.out.print("Enter name to update: ");
    String name = sc.nextLine();

    for (Contact c : contacts) {
        if (c.name.equalsIgnoreCase(name)) {
            System.out.print("Enter new phone: ");
            c.phone = sc.nextLine();
            System.out.print("Enter new address: ");
            c.address = sc.nextLine();
            System.out.println("Contact updated!");
            return;
        }
    }

    System.out.println("Contact not found!");
}

static void deleteContact(Scanner sc) {
    System.out.print("Enter name to delete: ");
    String name = sc.nextLine();
```

```
        Iterator<Contact> it = contacts.iterator();
        while (it.hasNext()) {
            Contact c = it.next();
            if (c.name.equalsIgnoreCase(name)) {
                it.remove();
                System.out.println("Contact deleted!");
                return;
            }
        }

        System.out.println("Contact not found!");
    }

    static void displayContacts() {
        if (contacts.isEmpty()) {
            System.out.println("No contacts available.");
            return;
        }

        for (Contact c : contacts) {
            System.out.println("\nName: " + c.name);
            System.out.println("Phone: " + c.phone);
            System.out.println("Address: " + c.address);
        }
    }
}
```

6. INPUT AND OUTPUT SCREEN

1: Main Menu Screen (Input)

Description:

This screen displays the main menu options of the Personal Contact Directory system.

--- CONTACT DIRECTORY ---

1. Add Contact

2. Search Contact

3. Update Contact

4. Delete Contact

5. Display All Contacts

6. Exit

Enter your choice: 1

2: Add Contact Screen (Input)

Description:

User enters contact details such as name, phone number, and address.

Enter Name: Riya

Enter Phone: 9876543210

Enter Address: Coimbatore

Contact added successfully!

3: Search Contact Screen (Output)

Description:

User searches for a contact using name.

Enter name to search: Riya

Name: Riya

Phone: 9876543210

Address: Coimbatore

4: Update Contact Screen (Input & Output)

Description:

User updates existing contact details.

Enter name to update: Riya

Enter new phone: 9999999999

Enter new address: Chennai

Contact updated!

5: Delete Contact Screen (Output)

Description:

User deletes a contact from the directory.

Enter name to delete: Riya

Contact deleted!

6: Display All Contacts (Output)

Description:

Displays all stored contacts.

Name: Anu

Phone: 8888888888

Address: Salem

Name: Ravi

Phone: 7777777777

Address: Erode

7: Exit Screen

Description:

Program exits when user selects option 6.

Exiting...

7. DISCUSSION & CONCLUSION

7.1 Discussion

The Personal Contact Directory with Search and Update Features was developed as a simple and efficient solution for managing personal contact information. The main purpose of this project was to overcome the limitations of manual contact management systems and provide a structured and user-friendly digital alternative.

During the development of this project, various Core Java concepts such as classes, objects, methods, loops, conditional statements, and ArrayList were used effectively. The modular design approach helped in dividing the system into smaller parts like adding, searching, updating, deleting, and displaying contacts, which made the program easier to understand and implement.

The system allows users to store contact details such as name, phone number, and address in an organized manner. The search functionality enables users to quickly find specific contacts without going through all records manually. Similarly, the update feature ensures that users can modify contact details whenever necessary, keeping the information accurate and up-to-date.

The delete functionality helps in removing unwanted or outdated contacts, which improves data organization and reduces redundancy. The display feature provides a complete overview of all stored contacts, making it easy for users to access information whenever required.

The program was tested with different inputs to ensure that all operations perform correctly. Test cases were used for adding new contacts, searching existing contacts, updating details, and deleting records. The results showed that the system works efficiently and produces accurate outputs.

One of the key advantages of this system is its simplicity and ease of use. Since it is a menu-driven application, even users with basic computer knowledge can operate it without difficulty. The system also reduces manual effort, saves time, and minimizes errors that may occur in traditional methods of contact management.

However, the system has some limitations. It does not use a database for permanent storage, so data may be lost once the program is closed (in basic version). It also does not include advanced features like security, login authentication, or cloud storage. Despite these limitations, the project successfully demonstrates the practical implementation of Java programming concepts.

Overall, this project provides a strong foundation for developing more advanced contact management systems in the future.

7.2 Conclusion

The Personal Contact Directory with Search and Update Features is a useful and practical application developed using Core Java. The project successfully achieves its main objective of providing an efficient system for managing personal contact information.

The system simplifies the process of storing, searching, updating, and deleting contact details. It reduces the time and effort required in manual contact management and ensures better organization of data. The use of a menu-driven interface makes the system user-friendly and easy to operate.

This project also helps in understanding how programming concepts can be applied to solve real-world problems. It demonstrates the importance of structured programming, modular design, and proper problem analysis in software development.

Although the system has some limitations, it can be further improved by adding advanced features such as:

• Database integration for permanent storage

• Graphical User Interface (GUI) for better user experience

• Login and security features

• Search using multiple fields (phone, address, etc.)

• Backup and recovery options

In conclusion, this project is a successful implementation of a simple contact management system and serves as a stepping stone for developing more complex and real-time applications in the future.

8. REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com